

Comunicaciones seguras en el internet de las cosas en redes 5G

Francisco Gago Vázquez
Universidad de Sevilla
Sevilla, España
fragagvaz@alum.us.es

Sergio García Eslava
Universidad de Sevilla
Sevilla, España
sergaesl@alum.us.es

Marco Antonio Herrera Luján
Universidad de Sevilla
Sevilla, España
marherluj@alum.us.es

Resumen

El despliegue de ciudades inteligentes requiere infraestructuras IoT eficientes, donde la tecnología Device-to-Device (D2D) sobre WiFi Direct juega un papel crucial para descongestionar y optimizar las redes 5G. Sin embargo, los protocolos estándar de WiFi Direct presentan vulnerabilidades críticas. Este trabajo se centra en localizar las fases vulnerables de la conexión y presentar los protocolos propuestos para mitigar los posibles ataques, garantizando la seguridad en el establecimiento de la conexión.

Palabras clave

IoT, Smart Cities, WiFi Direct, D2D, 5G, Ciberseguridad, Protocolo MAKE, SAS, RSSI, Commit/Open, Diffie-Hellman, DoS, Ciberseguridad, Autenticación mutua, Man-in-the-Middle, MAC (Message Authentication Code), MAC (Media Access Control), Nonces, Timestamp, Lifetime.

ÍNDICE

1.	Introducción	2
1.A.	Contribución a la investigación	2
2.	Contexto	2
2.A.	WiFi-Direct	2
2.B.	Protocolo de clave SAS	3
2.C.	Limitaciones WPS	3
3.	Objetivos de seguridad	4
3.A.	Requisitos funcionales de seguridad	4
3.B.	Modelo de adversario (Supuesto atacante)	4
4.	Protocolo propuesto	5
4.A.	Fase de descubrimiento	5
4.A.1.	Suposiciones	5
4.A.2.	Algoritmo	5
4.B.	Fase de Autenticación Mutua y Acuerdo de Clave	6
5.	Implementación y Demostración	7
5.A.	Escenario 1: Ataque DoS (Vulnerable)	7
5.B.	Escenario 2: Protección DoS (Propuesta)	8
5.C.	Escenario 3: Ataque MitM en protocolo estándar (Vulnerable)	10
5.D.	Escenario 4: Protección MitM (Protocolo MAKE)	11
6.	Análisis	12
6.A.	Análisis de rendimiento - Mitigación de ataques DoS	12
6.B.	Análisis de seguridad - Prevención MITM	12
6.C.	Comparativa arquitectónica	12
7.	Conclusiones	13
	Referencias	13

1. INTRODUCCIÓN

En la actualidad, se observa una evolución de las ciudades hacia soluciones más tecnológicas e inteligentes. Estas soluciones se aplican en diferentes contextos como la detección de niveles de basura en los contenedores, la monitorización de plazas de aparcamiento disponibles o la gestión eficiente de infraestructuras urbanas. Este avance se logra gracias a la integración de sensores electrónicos, móviles y sistemas de información a dispositivos físicos con la finalidad de optimizar las actividades cotidianas.

En este escenario, el internet de las cosas (IoT), transforma objetos convencionales en objetos inteligentes, convirtiéndose en el motor de las Smart Cities. [6] Como consecuencia, se prevé que para 2025 haya 75 mil millones de dispositivos IoT generando datos masivos. Para soportar este volumen de datos, se requiere una infraestructura con un gran ancho de banda y una muy baja latencia. Las redes 5G, ya que cumplen con estas características concretas y además permiten conexiones Device-to-Device (D2D), una tecnología crucial en este contexto. [1]

Las conexiones D2D son imprescindibles porque permiten a los dispositivos comunicarse directamente entre ellos sin la necesidad de pasar a través de una antena central que gestione las comunicaciones. Esto permite descongestionar la red y mejorar la eficiencia de la red. En redes 5G, estas comunicaciones pueden apoyarse tanto en servicios móviles como en Wi-Fi Direct. Sin embargo, el uso de los servicios móviles presenta limitaciones debido a la falta de cobertura en áreas urbanas. Por el otro lado, el WiFi Direct no tiene este problema, por lo que supone una ventaja significativa. Además, ofrece una transmisión de datos más rápida y menor latencia, siendo la opción más adecuada para implementar en las Smart Cities.

No obstante, el uso del WiFi Direct en conexiones D2D, tienen ciertos problemas de seguridad. [3] Al existir comunicación directa entre dispositivos, los atacantes pueden interferir en las comunicaciones de los dispositivos, permitiendo la escucha y modificación de mensajes y la suplantación de identidad. Las entidades están expuestas a diferentes ataques (Man-in-the-middle (MITM), DoS, etc..) lo que supone un riesgo importante. [5] Evidentemente, las entidades D2D requieren protección frente a este tipo de amenazas, haciendo necesario un protocolo seguro que evite estos problemas.

1.A. Contribución a la investigación

El artículo propone un protocolo de autenticación mutua y de establecimiento de claves seguro para el intercambio de información en comunicaciones Device-to-Device (D2D) sobre WiFi Direct en entornos IoT de ciudades inteligentes. La contribución de los autores se define en los siguientes cuatro fundamentos:

- **Eficiencia y robustez:** El protocolo debe ser ligero, eficiente y robusto. Esto se consigue a través del uso de métodos primitivos y ligeros de la criptografía como son el protocolo Commit/Open, la criptografía simétrica, el intercambio de claves Diffie-Hellman y códigos de autenticación de mensaje (MAC).
- **Mitigación temprana de DoS:** Los dispositivos deben poder realizar un filtrado inteligente sobre las diferentes peticiones de conexión para evitar posibles ataques DoS, evitando el malgasto de recursos en la fase de descubrimiento.
- **Validación de seguridad:** El protocolo debe demostrarse que funciona mediante el análisis de Lógica BAN (Burrows-Abadi-Needham), asegurando que realmente es seguro. Consiguiendo esto los autores exhiben la necesidad del uso de propiedades como la frescura del mensaje, la autenticación mutua, etc.
- **Integridad y verificación de claves:** Debe resistir ataques MITM, DoS y de repetición, y asegurar que ambas partes verifican que la clave acordada es la misma para evitar errores de cifrado/descifrado.

2. CONTEXTO

2.A. WiFi-Direct

El WiFi Direct es un protocolo de conexión de dispositivos, el cual permite a dos entidades conectarse sin la necesidad de un router externo. Esto permite una mayor transmisión de datos y una menor latencia, mejorando la eficiencia. Su funcionamiento se basa en las siguientes 4 fases (Fig. 1): [3]

1. **Fase de descubrimiento:** Es la fase inicial donde los dispositivos se detectan entre ellos para realizar las conexiones peer-to-peer (P2P). La búsqueda entre dispositivos se hace sin la necesidad de un router por el cual hacer las conexiones.
2. **Negociación de grupo:** Los dispositivos determinan quien actuará como Group Owner(GO), el cual es el que controla las comunicaciones del grupo. Este proceso se elige al GO, mediante un handshake, donde cada participante del grupo P2P envía un “valor de intención”. El dispositivo que posea el valor intención más alto es quien se convierte en GO del grupo P2P.
3. **Proceso WPS:** Se ejecuta el mecanismo de seguridad mediante WiFi Protected Setup (WPS), donde se lleva a cabo el acuerdo de claves y la autenticación mutua. Dentro del proceso de seguridad tradicional de WPS, el GO actúa como registrador para enviar las credenciales de seguridad de la red a las entidades del WiFi Direct. Tras esta fase, la protección del tráfico suele apoyarse en WPA (Wi-Fi Protected Access).

4. **Configuración de dirección:** Finalmente una vez autenticados, el GO asigna direcciones IP (típicamente vía DHCP) y se establece el canal D2D operativo.

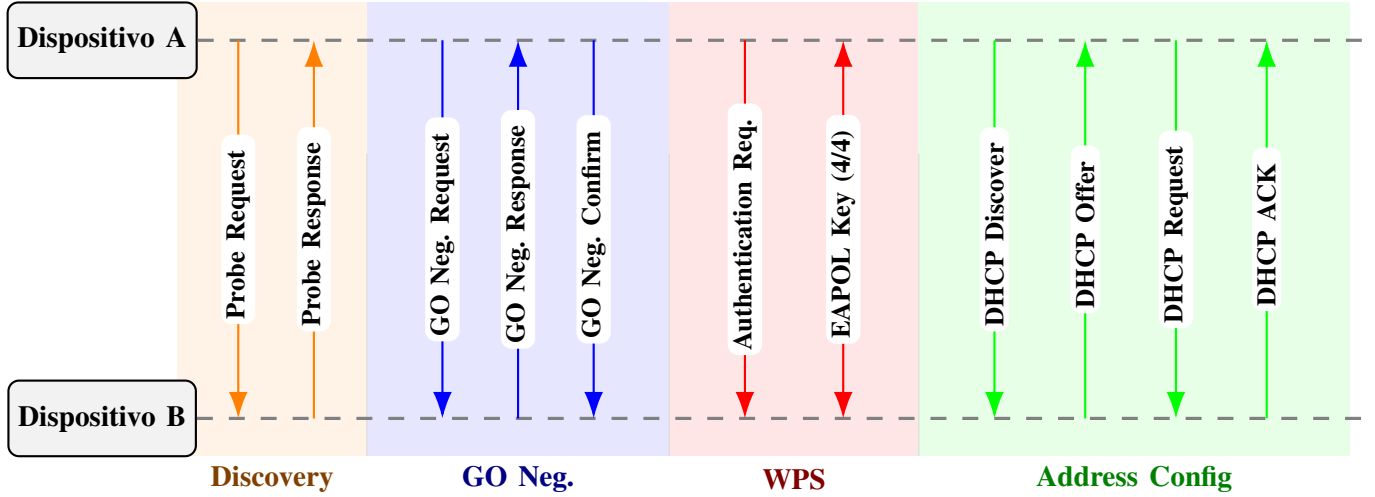


Figura 1: Diagrama de secuencia detallado de la conexión WiFi Direct estándar.

2.B. Protocolo de clave SAS

El protocolo Short Authentication String (SAS) es un acuerdo de clave diseñado para la autenticación mutua y el establecimiento de claves en las comunicaciones Device-to-Device (D2D). Se apoya en Diffie-Hellman como mecanismo fundamental para establecer una clave secreta.

Además, utiliza un sistema de intercambio de claves basado en un esquema de compromiso (commitment scheme). El proceso de funcionamiento, basando la comunicación entre Alicia y Bob (Dispositivos A y B), incluye los siguientes pasos clave:

1. **Generación de parámetros secretos:** Los dispositivos A y B generan sus identificadores (ID_x), parámetros Diffie Hellman (g^x) y números aleatorios: *nonces* (N_x). Con ello construyen sus valores secretos (m_x) concatenando su identificador, el parámetro Diffie-Hellman y el nonce:

$$m_x = ID_x \parallel g^x \parallel N_x \quad (1)$$

2. **Creación del Commit/Open Pair:** El dispositivo transforma el secreto m_x en un par (c, d) , donde c es el valor de compromiso y d el valor de apertura.
3. **Intercambio de Mensajes:**
 - Inicialmente, Alice comparte su compromiso (c) con Bob.
 - Como respuesta, Bob intercambia su valor secreto (m_B) con Alice.
 - Alice, tras recibir el secreto de Bob (m_B), envía su valor de apertura (d) permitiendo a Bob extraer el valor secreto de Alice (m_A).
4. **Cálculo y verificación (SAS):** Tras obtener ambos dispositivos los secretos (m_A, m_B), ambos verifican la identidad mutua calculando la Short Authentication String (SAS), aplicando XOR a N_A y N_B . Estas dos cadenas son enviadas a través de un canal confiable para ser comparadas. Si las cadenas de autenticación no coinciden ($S_A \neq S_B$), el proceso de establecimiento de claves se termina, asumiendo que existe un ataque Man-in-the-Middle (MITM).
5. **Establecimiento de la clave:** Si la verificación es exitosa, ambos dispositivos calculan la clave secreta:

$$K = g^{ab} \pmod{p} \quad (2)$$

A pesar de que el protocolo intenta prevenir el ataque MITM, este presenta una debilidad al compartir las cadenas de autenticación corta (SA, SB), ya que se comparte en texto simple a través de un canal inalámbrico vulnerable, permitiendo así el ataque MITM mediante la escucha (*eavesdropping*).

2.C. Limitaciones WPS

El protocolo WiFi Protected Setup (WPS) es un mecanismo de seguridad utilizado por WiFi Direct para proteger las comunicaciones D2D (Device-to-Device). Este permite que el Group Owner (GO) registre las comunicaciones, para enviar las credenciales de seguridad a los clientes. Además, ofrece protección mediante dos métodos principales: el PIN y el botón de configuración (Push Button). Ambos presentan limitaciones que comprometen la robustez del protocolo:

1. **Vulnerabilidad del PIN:** Se centra en un fallo del PIN de configuración, ya que este puede ser descubierto fácilmente mediante un ataque de fuerza bruta en una pequeña fracción de tiempo. [4]
2. **Vulnerabilidad del botón de configuración (Push Button):** En un escenario donde un atacante active su botón al mismo tiempo que el usuario, podría obtener acceso no autorizado al GO o al cliente. [4]

Estas dos vulnerabilidades comprometen la seguridad de las comunicaciones D2D, provocando accesos y revelación de secretos en las comunicaciones.

3. OBJETIVOS DE SEGURIDAD

El diseño de un protocolo criptográfico para infraestructuras críticas en Smart Cities debe resolver la tensión entre la necesidad de una seguridad robusta y las severas limitaciones de hardware de los dispositivos IoT. Para garantizar la viabilidad y confiabilidad del sistema propuesto, hemos formalizado los objetivos de seguridad basándonos en tres pilares fundamentales: [1], [5]

- **Disponibilidad:** El sistema debe garantizar la continuidad del servicio incluso en presencia de ataques activos de saturación. En el contexto de un semáforo inteligente, la incapacidad de procesar datos legítimos debido a un ataque DoS podría derivar en accidentes de tráfico o congestión urbana.
- **Integridad y autenticidad:** Se debe asegurar matemáticamente que la información recibida proviene de una fuente legítima y no ha sido manipulada en tránsito. La veracidad del dato es crítica; un atacante no debe poder inyectar estados de tráfico falsos.
- **Eficiencia computacional:** Debido a que los nodos IoT operan a menudo con baterías y CPUs de bajo consumo, los mecanismos de seguridad no deben introducir latencias que degraden el servicio ni agotar las reservas de energía prematuramente.

3.A. Requisitos funcionales de seguridad

Para satisfacer los objetivos anteriores, el protocolo debe cumplir con los siguientes requisitos criptográficos específicos:

- **Autenticación mutua en D2D:** A diferencia de las redes centralizadas con una autoridad confiable, en WiFi Direct el canal es abierto. Es imperativo que tanto el emisor como el receptor verifiquen la identidad el uno del otro antes de intercambiar información sensible, evitando conexiones con nodos suplantadores.
- **Confidencialidad y acuerdo de claves:** El protocolo debe establecer una clave de sesión única y efímera (K_{ab}) derivada de parámetros Diffie-Hellman. Esto garantiza que, aunque un atacante capture el tráfico cifrado, no podrá descifrar el contenido sin resolver el problema del logaritmo discreto.
- **Integridad de datos:** Se requiere un mecanismo de vinculación criptográfica fuerte. Cualquier alteración en el payload (carga útil) del mensaje debe invalidar la verificación de la firma o el código MAC (Message Authentication Code), provocando el descarte inmediato del paquete.
- **Frescura del mensaje:** El sistema debe ser inmune a ataques de repetición (*Replay Attacks*). Un atacante no debe poder grabar un mensaje legítimo antiguo y reinyectarlo en la red horas después para causar disrupción. Esto requiere el uso de nonces aleatorios y marcas de tiempo (timestamps).
- **Resistencia a DoS en fase de descubrimiento:** El protocolo debe incorporar mecanismos de filtrado temprano para descartar solicitudes maliciosas antes de que consuman recursos de procesamiento criptográfico costoso.

Aplicación al escenario de estudio: Traducido a nuestro entorno de simulación, estos requisitos implican que el semáforo solo aceptará conexiones de cámaras que puedan probar posesión de un secreto compartido, que el estado del tráfico viaja cifrado para evitar espionaje, y que cualquier intento de inyectar una señal grabada previamente será rechazado por caducidad temporal.

3.B. Modelo de adversario (Supuesto atacante)

Para el análisis formal de seguridad, asumimos un adversario activo bajo el modelo estándar de Dolev-Yao. Este modelo, ampliamente aceptado en la verificación de protocolos, establece que "la red es el atacante". Las capacidades atribuidas al adversario son:

- **Control total del canal:** El atacante puede escuchar (*eavesdrop*), interceptar, modificar, inyectar y reordenar cualquier mensaje transmitido por el canal inalámbrico.
- **Asimetría de recursos:** A diferencia de los dispositivos IoT legítimos, asumimos que el atacante dispone de recursos computacionales (CPU/RAM) y energéticos significativamente superiores. Esto le permite ejecutar ataques de fuerza bruta limitados o mantener ataques de inundación (*flooding*) sostenidos en el tiempo.
- **Conocimiento del protocolo:** Siguiendo el principio de Kerckhoffs, asumimos que el atacante conoce perfectamente los algoritmos y el funcionamiento del protocolo público, desconociendo únicamente las claves privadas y los números aleatorios (nonces) generados dinámicamente.
- **Objetivos Maliciosos:** El adversario busca tanto la interrupción del servicio (Ataques DoS para agotar batería o CPU) como la subversión de la comunicación (Ataques MITM para suplantar al semáforo o la cámara).

4. PROTOCOLO PROPUESTO

Las fases más propensas a ataques son Descubrimiento y Autenticación, como se muestra en la Fig. 2. Por lo tanto, el protocolo descrito en el artículo propone una serie de soluciones a los problemas conocidos en estas fases vulnerables.

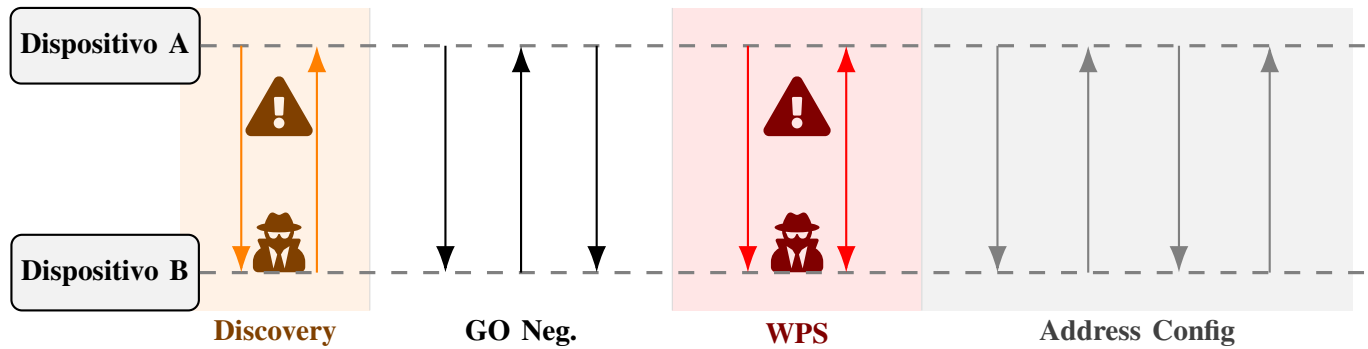


Figura 2: Etapas críticas en la conexión WiFi Direct estándar.

4.A. Fase de descubrimiento

Los dispositivos que utilizan Wifi Direct necesitan enviar una solicitud *probe request* para conectarse con otro dispositivo, por lo que los dispositivos que utilizan Wifi Direct deben permanecer escuchando (es decir, a la espera de una *probe request*) para responder con una *probe response* esa solicitud de conexión. Esta característica puede explotarse de forma maliciosa, ya que es posible crear *probe requests* falsas con la única finalidad de impedir el correcto funcionamiento del dispositivo objetivo. Esto se conoce como un ataque de Denegación de Servicio (DoS). El protocolo propuesto expone algunas medidas para hacer frente a esta vulnerabilidad. En concreto, se plantea un mecanismo de filtrado en la fase de descubrimiento mediante el uso de un algoritmo.

4.A.1. *Suposiciones:* Antes de introducir el algoritmo, se realizan una serie de asunciones para acotar el problema:

- La dirección MAC (o dirección física de un dispositivo) es única, por lo que no puede haber dos dispositivos legítimos con la misma dirección MAC.
- Los dispositivos emiten señales con una intensidad que puede ser medida por el receptor en forma de RSSI (Received Signal Strength Indicator), el cual sirve para estimar la localización del dispositivo que emite la señal.
- Los dispositivos han sido instalados de forma distribuida y estacionaria (ningún dispositivo se mueve según esta suposición).
- La dirección MAC y el RSSI sirven para distinguir a un dispositivo de otro.
- Los dispositivos tienen capacidad de almacenamiento para guardar las direcciones MAC y los valores RSSI de los dispositivos que han solicitado conexión previamente.
- Los factores que podrían afectar al RSSI se consideran constantes.

4.A.2. *Algoritmo:* El algoritmo (Fig. 15) asegura que no se procesen solicitudes de dispositivos con la misma MAC y mismo RSSI, o distinta MAC y mismo RSSI.

Algoritmo 1 Filtrado de solicitudes maliciosas en la fase de descubrimiento

Entrada: Dirección MAC, RSSI**Salida:** Recursos, DoS**Abreviaturas:** new probe request (NPR), compares with database (CWD)

```
1:  $D_1$  envía_solicitud  $D_2$                                 ▷  $D_1$  envía solicitud de sondeo a  $D_2$ 
2:  $D_2$  recibe_solicitud  $D_1$                                 ▷  $D_2$  almacena  $MAC^1, RSSI^1$ 
3: if  $MAC^2 == MAC^1$  then                                    ▷  $MAC^2, RSSI^2 \in NPR$ 
4:   if  $RSSI^2 == RSSI^1$  then                                ▷  $D_2$  realiza CWD
5:     DoS()                                                  ▷ Denegación de servicio
6:   end if
7: else
8:   if  $MAC^2 \neq MAC^1$  then
9:     if  $RSSI^2 \neq RSSI^1$  then
10:      proveer_servicio()                                    ▷ Recursos concedidos
11:    end if
12:  else
13:    DoS()                                                    ▷ Denegación de servicio
14:  end if
15: end if
```

4.B. Fase de Autenticación Mutua y Acuerdo de Clave

Wifi Direct emplea un mecanismo de intercambio de claves que permite que un posible adversario o *hacker* pueda interponerse en este intercambio y modificar los siguientes mensajes enviados entre los dispositivos. Esto se conoce como un ataque *Man in the Middle (MITM)*. El protocolo propuesto emplea un esquema llamado MAKE (Mutual Authentication and Key Agreement) para prevenir el ataque *MITM*, el cual pretende mejorar el acuerdo de clave SAS arreglando sus puntos débiles (Fig. 14). Con este fin, se realizan las siguientes mejoras:

- Se añaden los valores Timestamp (TS) y Lifetime (LT).
- El intercambio de Short Authentication Strings (SAS) no se realiza en texto plano.
- Verificación de la generación de claves en ambos dispositivos.

Algoritmo 2 MAKE - Mutual Authentication and Secret Key Establishment

Entrada: IDs de dispositivos capturadas en fase de descubrimiento: ID_A, ID_B ; Claves públicas: g, p ; Cada dispositivo elige un valor secreto (s) y calcula $g^s \pmod{p}$ ($g^a \pmod{p}, g^b \pmod{p}$); Cálculo de Nonce N_A y N_B en disp. A y B respectivamente; Message Authentication Code (MAC); Timestamp (TS); Lifetime (LT)**Salida:** MAKE Exitoso, MAKE Fallido

```
1:  $D_B \leftarrow c$                                 ▷ Disp. A envía valor de compromiso al Disp. B
2:  $D_A \leftarrow m_B$                                 ▷ Disp. B envía  $m_B$  al Disp. A
3:  $D_B \leftarrow d$                                 ▷ Disp. A envía valor de apertura al Disp. B
4:  $S_A = N_A \oplus N'_B$                                 ▷ Disp. A calcula cadena de autenticación
5:  $S_B = N'_A \oplus N_B$                                 ▷ Disp. B calcula cadena de autenticación
6:  $K_{AB} = (g^b)^a \pmod{p}$                                 ▷ Disp. A calcula clave secreta
7:  $K_{BA} = (g^a)^b \pmod{p}$                                 ▷ Disp. B calcula clave secreta
8:  $D_B \leftarrow Auth_A = TS || LT || MAC_A(K_{AB}, S_A || TS || LT)$ 
9:  $D_A \leftarrow Auth_B = MAC_B(K_{BA}, S_B || TS || LT)$ 
10: if  $MAC_A == MAC_B$  then                                ▷  $D_A$  y  $D_B$  verifican
11:   MAKE Exitoso
12: else
13:   MAKE Fallido
14: end if
```

En el caso en el que A se quiere conectar con B sucede lo siguiente:

1. Los dispositivos conocen:
 - Su propio ID (ID_A, ID_B)
 - El ID del otro dispositivo
 - Los parámetros g y p del intercambio de claves Diffie-Hellman
2. Cada dispositivo genera:

- a o b : valor aleatorio (lo llamaremos x de forma genérica)
 - g^x (mód p)
 - N_x : valor “nonce” entre 0 y 1
 - $m_x = ID_x \parallel g^x \parallel N_x$
3. A genera un par “commit-open” (c, d) a partir de m_A
 4. A envía c a B
 5. B envía m_B a A
 6. A envía d a B
 7. Se calculan los String de autenticación (SAS) y las claves secretas de Diffie-Hellman:
 - $S_A = N_A \oplus N_B$
 - $S_B = N_A \oplus N_B$
 - $K_{AB} = (g^b)^a$ (mód p)
 - $K_{BA} = (g^a)^b$ (mód p)
 8. Se envían las confirmaciones de la autenticación y verificación de que utilizan la misma clave secreta y añadiendo valores de Timestamp (TS), es decir, cuándo se realizó el envío y de Lifetime (LT), es decir, hasta cuándo es válido este valor:
 - $Auth_A = MAC(K_{AB}, S_A \parallel TS \parallel LT)$
 - $Auth_B = MAC(K_{BA}, S_B \parallel TS \parallel LT)$

Si los valores intercambiados por Message Authentication Code (MAC) coinciden, se concluye como intercambio de claves y autenticación exitosa.

5. IMPLEMENTACIÓN Y DEMOSTRACIÓN

Para validar la viabilidad práctica de los protocolos teóricos presentados, se ha desarrollado un simulador interactivo basado en Python (FastAPI) y una interfaz web reactiva. [8] Este entorno permite reproducir ataques reales y visualizar la eficacia de las contramedidas en tiempo real. Se adjunta al proyecto un *Manual de Usuario* detallado para la ejecución del software. [7]

A continuación, se documentan los resultados obtenidos en los cuatro escenarios de prueba disponibles en la herramienta.

5.A. Escenario 1: Ataque DoS (Vulnerable)

En este modo, se simula un dispositivo IoT receptor (Cámara) sin mecanismos de filtrado previo. El objetivo es demostrar la fragilidad de los dispositivos con recursos limitados ante solicitudes masivas.

El proceso se inicia cuando el atacante (Hugo) lanza un ataque de inundación (*flooding*). [2] Como se observa en la Fig. 3, el adversario envía una ráfaga continua de paquetes de solicitud (REQ) hacia el dispositivo víctima, saturando el canal de comunicación.

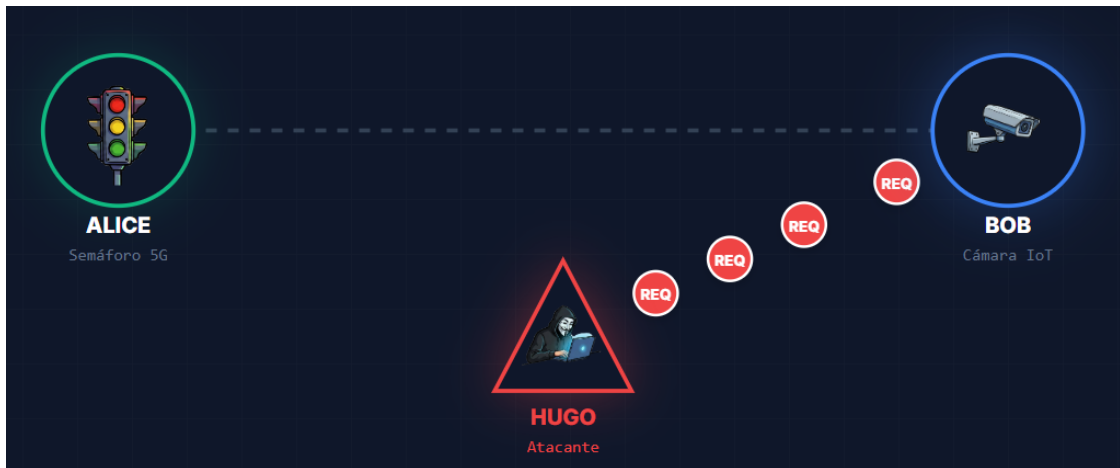


Figura 3: Inicio del ataque: El adversario inyecta múltiples solicitudes de conexión simultáneas hacia la víctima.

Al carecer de un algoritmo de validación en la fase de descubrimiento, la víctima intenta procesar cada una de estas solicitudes como si fueran legítimas. La Fig. 4 muestra los registros internos del sistema durante el ataque: inicialmente se aceptan los paquetes, pero la carga computacional escala rápidamente hasta agotar la capacidad de procesamiento, momento en el que produce un fallo del sistema.

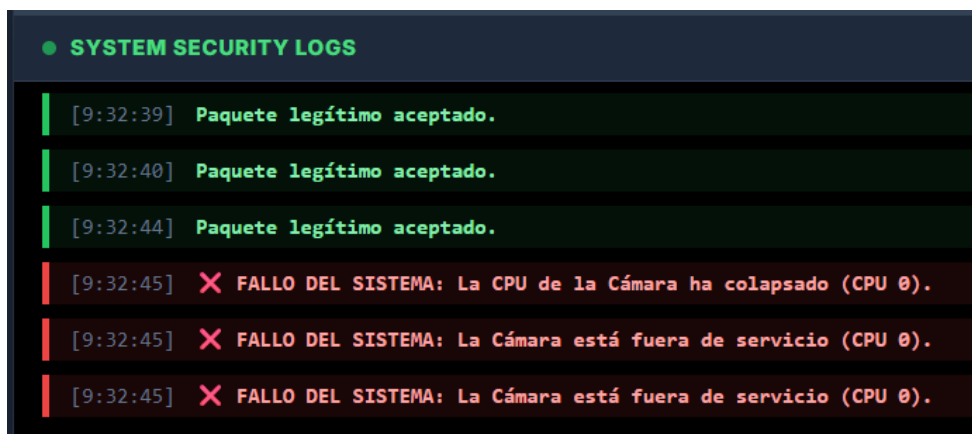


Figura 4: Logs del sistema: La aceptación de todos paquetes lleva a la CPU a 0, provocando el fallo crítico.

La consecuencia final se ilustra en la Fig. 5. Una vez que la CPU alcanza el colapso total (0 %), el dispositivo queda inoperativo (representado por el icono de la cámara dañada), denegando el servicio a cualquier usuario legítimo que intente conectarse posteriormente.

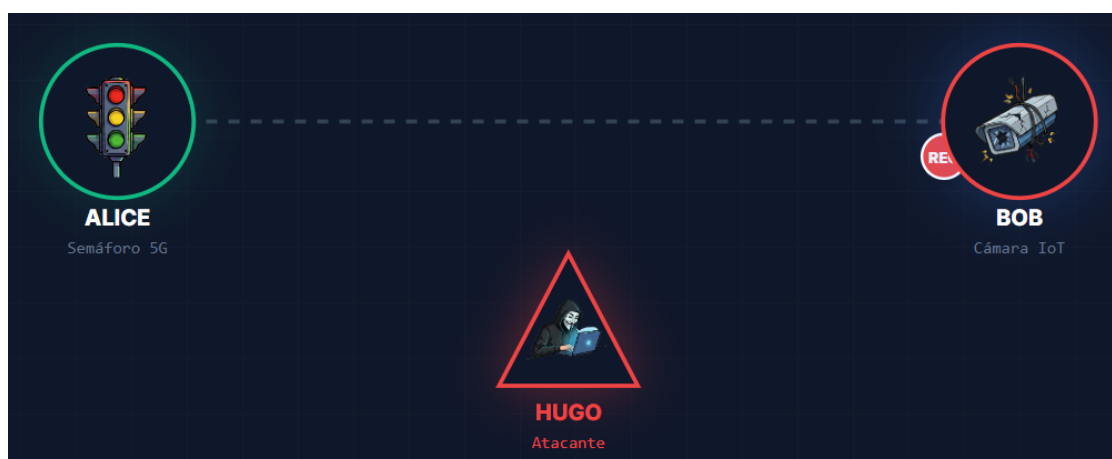


Figura 5: Colapso del nodo: El dispositivo víctima queda fuera de servicio tras el agotamiento de recursos.

5.B. Escenario 2: Protección DoS (Propuesta)

Para validar la robustez del algoritmo de filtrado contextual, se ha diseñado una prueba secuencial de cinco fases que evalúa la capacidad del sistema para distinguir entre usuarios legítimos y atacantes basándose en la identidad (MAC) y el contexto físico (RSSI).

En primer lugar, el dispositivo legítimo (Semáforo) solicita acceso con sus credenciales reales (Fig. 6: MAC terminada en :66) y una potencia de señal de -45 dBm. La Fig. 7 muestra cómo el sistema verifica la solicitud y concede el acceso, estableciendo una sesión activa.

QUIÉN ENVÍA:

SEMÁFORO HACKER

MAC ADDRESS

11:22:33:44:55:66

RSSI (-45 dBm)

ENVIAR PAQUETE

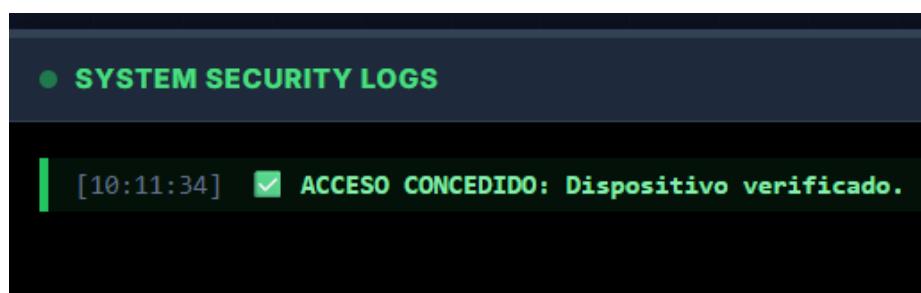


Figura 7: Resultado: Dispositivo verificado.

Figura 6: Solicitud legítima.

El atacante (Hugo el Hacker) intenta suplantar al semáforo copiando exactamente sus parámetros (Fig. 8): misma dirección MAC y mismo RSSI (-45 dBm). Como se observa en la Fig. 9, el protocolo detecta que esa MAC ya tiene una sesión activa registrada y bloquea inmediatamente el intento, protegiendo la integridad de la sesión en curso.

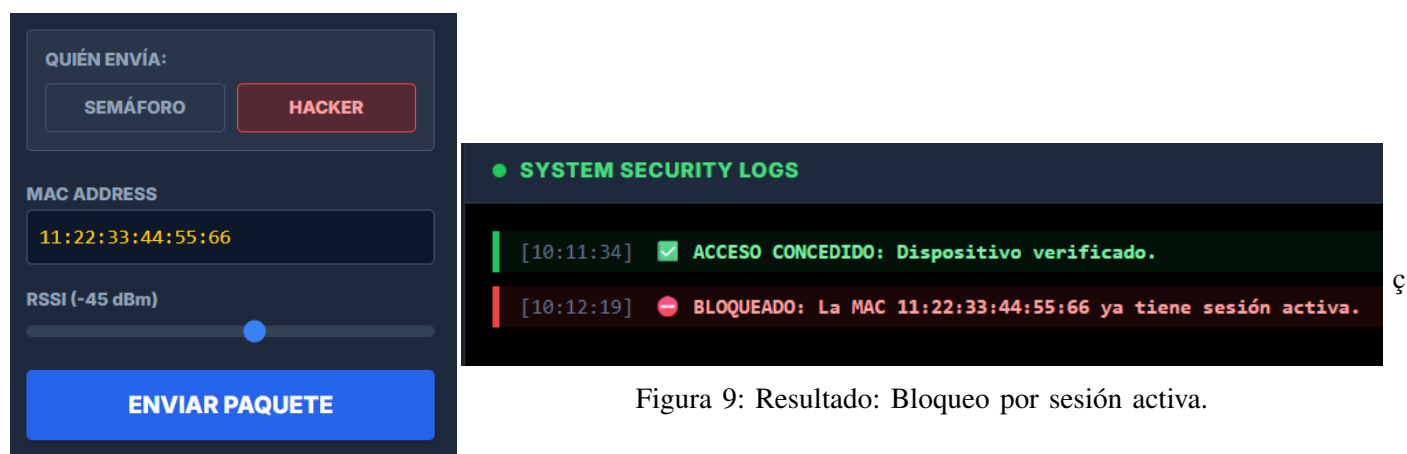


Figura 8: Intento con misma MAC y RSSI.

El atacante modifica su estrategia: mantiene la MAC suplantada pero altera la potencia de emisión a -67 dBm (Fig. 10), intentando simular un cambio de posición o fluctuación. La Fig. 11 evidencia que el sistema no se deja engañar; al persistir el uso de una MAC ya registrada en el sistema, la solicitud sigue siendo rechazada independientemente del cambio en la señal.

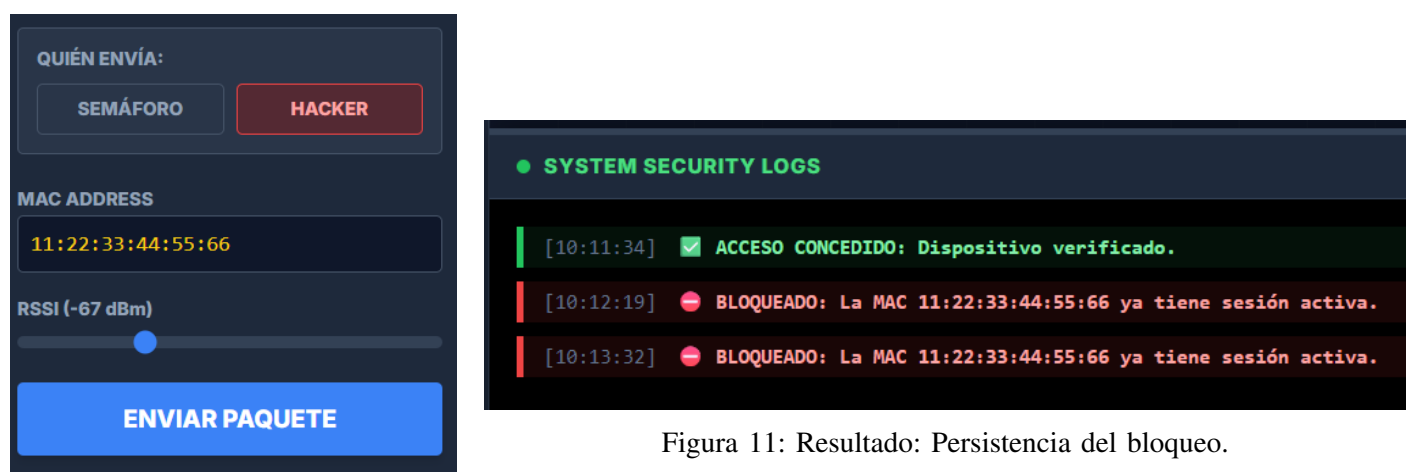


Figura 10: Misma MAC con RSSI modificado.

En esta fase, el atacante cambia su identidad (Fig. 12), MAC nueva terminada en :FF, pero vuelve a emitir con la potencia original del semáforo (-45 dBm). El algoritmo de filtrado contextual identifica una anomalía física: es improbable que dos dispositivos distintos ocupen exactamente el mismo espacio radioeléctrico (mismo RSSI exacto). Por tanto, como muestra la Fig. 13, se bloquea la solicitud por "Distinta MAC en misma ubicación".

QUIÉN ENVÍA:

SEMÁFORO
HACKER

MAC ADDRESS

AA:BB:CC:DD:EE:FF

RSSI (-45 dBm)

ENVIAR PAQUETE

Figura 12: Nueva MAC con RSSI ya almacenado.

Finalmente, se simula una conexión válida donde cambian tanto la identidad como el contexto. El dispositivo presenta una MAC nueva y un RSSI diferente (-19 dBm) (Fig. 14), lo cual es coherente con un nuevo nodo en la red. La Fig. 15 confirma que el sistema valida estos parámetros al no encontrar conflictos, registrando exitosamente el nuevo dispositivo.

QUIÉN ENVÍA:

SEMÁFORO
HACKER

MAC ADDRESS

AA:BB:CC:DD:EE:FF

RSSI (-19 dBm)

ENVIAR PAQUETE

Figura 14: Cambios en MAC y RSSI.

SYSTEM SECURITY LOGS

[10:11:34]
ACCESO CONCEDIDO: Dispositivo verificado.

[10:12:19]
BLOQUEADO: La MAC 11:22:33:44:55:66 ya tiene sesión activa.

[10:13:32]
BLOQUEADO: La MAC 11:22:33:44:55:66 ya tiene sesión activa.

[10:15:26]
BLOQUEADO: Distinta MAC en misma ubicación.

Figura 13: Bloqueo por tener misma MAC en distinta ubicación.

SYSTEM SECURITY LOGS

[10:11:34]
ACCESO CONCEDIDO: Dispositivo verificado.

[10:12:19]
BLOQUEADO: La MAC 11:22:33:44:55:66 ya tiene sesión activa.

[10:13:32]
BLOQUEADO: La MAC 11:22:33:44:55:66 ya tiene sesión activa.

[10:15:26]
BLOQUEADO: Distinta MAC en misma ubicación.

[10:16:43]
NUEVO DISPOSITIVO: Registrado exitosamente.

Figura 15: Resultado: Registro exitoso.

5.C. Escenario 3: Ataque MitM en protocolo estándar (Vulnerable)

En este escenario se evalúa la resistencia del protocolo estándar SAS frente a un atacante activo (Hugo). Para contextualizar el impacto del ataque, primero se presenta el funcionamiento nominal del sistema y posteriormente el ataque exitoso del Hacker.

Conexión legítima (Sin ataque): En condiciones normales, como se observa en la Fig. 16, Alice y Bob intercambian sus claves públicas Diffie-Hellman sin interferencias. El log confirma que ambos calculan la misma clave de sesión ($S_A = S_B = 11871$), completando la autenticación mutua de forma exitosa.

SYSTEM SECURITY LOGS

[0:09:54]
Parámetros Diffie-Hellman válidos: p = 23 (primo), g = 5 (generador)

[0:09:55]
Alice -> Bob [M1]: c (commit) [c = hash(AA:BB:CC:DD:EE:01||13||2812), mA = AA:BB:CC:DD:EE:01 || 13 || 2812]

[0:09:58]
Bob -> Alice [M2]: mB = IDB || gb || NB [mB = AA:BB:CC:DD:EE:02 || 3 || 9379]

[0:10:00]
Alice -> Bob [M3]: dA => mA = IDA || ga || NA [dA = open(hash(AA:BB:CC:DD:EE:01||13||2812)) => AA:BB:CC:DD:EE:01||13||2812, c = hash(AA:BB:CC:DD:EE:01||13||2812)]

[0:10:02]
Autenticación mutua completada: SA = SB = NA ⊕ NB [SA = 11871, SB = 11871]

Figura 16: Ejecución nominal: Alice y Bob derivan claves idénticas ($S_A = S_B$) sin interferencias.

Ejecución del Ataque MitM: La vulnerabilidad crítica se manifiesta cuando el atacante intercepta el canal. Como se evidencia en las Figs. 17 y 18, Hugo se posiciona entre las víctimas:

1. **Intercepción:** Hugo detiene los mensajes de compromiso ($M1, M2$) y reemplaza las claves públicas (g_a, g_b) por las suyas propias (g_e).
2. **Suplantación:** Alice cree establecer una clave con Bob, pero en realidad establece $K_{AE} = 22$ con Hugo. Simultáneamente, Bob establece $K_{BE} = 20$ con Hugo.
3. **Resultado:** El log final indica MITM COMPLETADO. Hugo posee ambas claves y puede descifrar y re-cifrar el tráfico entre Alice y Bob de manera transparente, comprometiendo totalmente la confidencialidad.

```

● SYSTEM SECURITY LOGS

[0:11:12] ✅ Parámetros Diffie-Hellman válidos: p = 23 (primo), g = 5 (generador)
[0:11:12] 📧 Alice -> Hugo [M1]: c (Hugo intercepta) [c = hash(AA:BB:CC:DD:EE:01||22|6181), mA = AA:BB:CC:DD:EE:01 || 22 || 6181]
[0:11:15] ⚠️ Hugo -> Bob [M2]: ce (Hugo suplanta a Alice) [ce = hash(AA:BB:CC:DD:EE:01||14|7845), mE = AA:BB:CC:DD:EE:01 || 14 || 7845]
[0:11:16] 📧 Bob -> Hugo [M3]: mB = IDB || gb || NB [mB = AA:BB:CC:DD:EE:02 || 15 || 1197]
[0:11:18] ⚠️ Hugo -> Alice [M4]: mB' = IDB || ge || NE (Hugo suplanta a Bob) [mBp = AA:BB:CC:DD:EE:02 || 14 || 7845, KAE = 22]
[0:11:20] 📧 Alice -> Hugo [M5]: dA => mA = IDA || ga || NA [dA = open(hash(AA:BB:CC:DD:EE:01||22|6181)) => AA:BB:CC:DD:EE:01||22|6181, c = hash(AA:BB:CC:DD:EE:01||22|6181)]

```

Figura 17: Fase inicial del ataque: Hugo intercepta los parámetros y suplanta las identidades de Alice y Bob.

```

● SYSTEM SECURITY LOGS

hash(AA:BB:CC:DD:EE:01||22|6181)
[0:11:22] ⚠️ Hugo -> Bob [M6]: dE => mE = IDA || ge || NE [dE = open(hash(AA:BB:CC:DD:EE:01||14|7845)) => AA:BB:CC:DD:EE:01||14|7845, ce = hash(AA:BB:CC:DD:EE:01||14|7845)]
[0:11:42] 📧 Alice -> Hugo [M7]: SA = NA ⊕ NE [SA = 1664]
[0:11:43] 📧 Bob -> Hugo [M8]: SB = NB ⊕ NE [SB = 6664]
[0:11:47] ⚠️ Hugo -> Alice [M9]: SAE = NA ⊕ NE [SAE = 1664]
[0:11:49] ⚠️ Hugo -> Bob [M10]: SBE = NB ⊕ NE - MITM COMPLETADO [SBE = 6664, KAE = 22, KBE = 20]

```

Figura 18: Ataque exitoso: Se establecen dos claves de sesión distintas ($S_{AE} \neq S_{BE}$), otorgando al atacante control total del canal.

5.D. Escenario 4: Protección MitM (Protocolo MAKE)

Finalmente, se valida la eficacia de la propuesta MAKE. Este protocolo introduce verificación de integridad mediante MAC y sellado de tiempo para prevenir la manipulación de parámetros en tránsito.

Como se muestra en la secuencia de las Fig. 19, 20 y 21, el atacante intenta replicar la estrategia anterior modificando el mensaje $M4$ para inyectar su clave (g_e). Sin embargo, el mecanismo de seguridad responde de la siguiente manera:

- El protocolo detecta que la clave derivada por el atacante (K_{AE}) no coincide con la esperada por la víctima (K_{BE}).
- Al llegar al paso de verificación de autenticación (Mensaje $M11/M12$), Hugo es incapaz de falsificar un MAC válido porque desconoce el secreto compartido original.
- **Resultado:** El sistema lanza la alerta BLOQUEADO [M12]. La conexión se aborta inmediatamente, protegiendo a los usuarios de establecer un canal inseguro.

```

● SYSTEM SECURITY LOGS
LIMPIAR CONSOLA

[1:43:57] ✅ Parámetros Diffie-Hellman válidos: p = 23 (primo), g = 5 (generador)
[1:43:58] 📧 Alice -> Hugo [M1]: cA (Hugo intercepta) [cA = hash(AA:BB:CC:DD:EE:01||18|5693), mA = AA:BB:CC:DD:EE:01 || 18 || 5693]
[1:43:59] ⚠️ Hugo -> Bob [M2]: cE (Hugo suplanta a Alice) [cE = hash(AA:BB:CC:DD:EE:01||8|3163), mE = AA:BB:CC:DD:EE:01 || 8 || 3163]
[1:44:00] 📧 Bob -> Hugo [M3]: mB = IDB || gb || NB [mB = AA:BB:CC:DD:EE:02 || 14 || 1570]
[1:44:01] ⚠️ Hugo modifica mB [M4]: mB' = IDB || ge || NB (reemplaza gb con ge) [mB = AA:BB:CC:DD:EE:02 || 14 || 1570, mBp = AA:BB:CC:DD:EE:02 || 8 || 1570]
[1:44:03] ⚠️ Hugo -> Alice [M5]: mB' (Hugo suplanta a Bob) [mBp = AA:BB:CC:DD:EE:02 || 8 || 1570]

```

Figura 19: Fase de Inyección: El atacante intercepta el intercambio Diffie-Hellman e intenta manipular el mensaje $M4$ reemplazando la clave pública legítima (g_b) por la suya (g_e).

```
● SYSTEM SECURITY LOGS [LIMPIAR CONSOLA]

[1:44:32] Alice -> Hugo [M6]: dA => mA = IDA || ga || NA [dA = open(hash(AA:BB:CC:DD:EE:01||18||5693)) => AA:BB:CC:DD:EE:01||18||5693, cA = hash(AA:BB:CC:DD:EE:01||18||5693)]
[1:44:33] Hugo -> Bob [M7]: dE => mE = IDA || ge || NE [dE = open(hash(AA:BB:CC:DD:EE:01||8||3163)) => AA:BB:CC:DD:EE:01||8||3163, cE = hash(AA:BB:CC:DD:EE:01||8||3163)]
[1:44:34] Alice calcula [M8]: SA = NA ⊕ NB = 4127 [SA = 4127]
[1:44:35] Bob calcula [M9]: SB = NA ⊕ NB = 4127 [SB = 4127]
[1:44:37] Alice -> Hugo [M10]: AuthA = 1766105037 || 3600 || MAC(KAE, 4127 || 1766105037 || 3600) [SA = 4127, KAE = 8, TS = 1766105037, LT = 3600, AuthA = MAC(8, 4127 || 1766105037 || 3600)]
```

Figura 20: Desincronización de Claves: Como resultado de la manipulación, las víctimas derivan claves de sesión divergentes ($S_A \neq S_B$) que el atacante no puede unificar.

```
● SYSTEM SECURITY LOGS [LIMPIAR CONSOLA]

[1:45:09] Bob -> Hugo [M11]: AuthB = MAC(KBE, 4127 || 1766105037 || 3600) [SB = 4127, KBE = 3, TS = 1766105037, LT = 3600, AuthB = MAC(3, 4127 || 1766105037 || 3600)]
[1:45:10] BLOQUEADO [M12]: Hugo no puede falsificar el MAC correcto. KAE ≠ KBE. MITM detectado y prevenido. [result = ✅ MITM detectado y prevenido, reason = Hugo no puede crear MACs válidos porque KAE ≠ KBE, KAE = 8, KBE = 3, TS = 1766105037, LT = 3600]
```

Figura 21: Prevención del Ataque: El mecanismo de seguridad verifica el MAC en el paso M12. Al detectar la firma inválida (debido a la clave incorrecta del atacante), el sistema bloquea la conexión instantáneamente.

6. ANÁLISIS

En esta sección se evalúa formalmente la robustez y eficiencia del protocolo propuesto, contrastando sus mecanismos de defensa con las vulnerabilidades inherentes al estándar WiFi Direct actual.

6.A. Análisis de rendimiento - Mitigación de ataques DoS

La principal diferencia en el rendimiento radica en la gestión de recursos durante la fase de descubrimiento.

- **Protocolo estándar (Vulnerable):** El estándar actual carece de mecanismos de validación previa en la capa de descubrimiento. Esto obliga al dispositivo a asignar recursos de memoria y procesamiento a cualquier solicitud entrante (*Probe Request*), independientemente de su legitimidad. Esta arquitectura es intrínsecamente susceptible a ataques de inundación, donde la saturación de la pila de protocolos provoca la indisponibilidad del servicio.
- **Protocolo propuesto (Eficiente):** La propuesta introduce una capa de abstracción que realiza un filtrado contextual basado en identificadores físicos y métricas de señal (RSSI). Teóricamente, esto permite descartar solicitudes maliciosas o repetitivas de forma inmediata y ligera, evitando que alcancen las capas superiores de procesamiento criptográfico y preservando así la estabilidad del sistema.

6.B. Análisis de seguridad - Prevención MITM

La seguridad del protocolo se fundamenta en la imposibilidad matemática de alterar los parámetros de negociación sin ser detectado.

- **Protocolo estándar (Vulnerabilidad SAS):** Los protocolos basados en *Short Authentication String* (SAS) tradicionales envían los compromisos o las cadenas de autenticación en claro o sin vinculación temporal fuerte. Un atacante con control del canal puede interceptar el intercambio y modificar los parámetros Diffie-Hellman al vuelo, estableciendo claves de sesión fraudulentas sin que las partes legítimas perciban la intrusión.
- **Protocolo propuesto (Integridad criptográfica):** El uso del esquema *Commit/Open* obliga al iniciador a comprometerse con un valor oculto (hash) antes de revelar la clave pública. Esto elimina la ventana de oportunidad para ataques adaptativos. Además, la inclusión de marcas de tiempo (*Timestamp*) y tiempo de vida (*Lifetime*) en la firma MAC garantiza la frescura del mensaje, neutralizando teóricamente los ataques de repetición.

6.C. Comparativa arquitectónica

La Tabla 1 resume las diferencias estructurales y sus implicaciones directas en la seguridad del sistema.

Tabla 1: Comparativa: Estándar vs. Propuesta

Propiedad	Estándar WiFi Direct	Protocolo propuesto
Gestión de solicitudes	Procesamiento indiscriminado de cualquier petición.	Filtrado inteligente basado en el contexto.
Impacto DoS	Agotamiento de recursos (CPU y Batería). El sistema falla.	Rechazo inmediato. El sistema se mantiene estable.
Intercambio de claves	Vulnerable a ataques de intermediario (MITM).	Seguro gracias a la vinculación fuerte (Commit/Open).
Integridad	Verificación visual (el usuario compara códigos).	Verificación matemática automática.

7. CONCLUSIONES

El crecimiento exponencial de las Smart Cities plantea desafíos críticos de seguridad que no pueden ser resueltos mediante la infraestructura tradicional. Este trabajo ha demostrado que es posible asegurar las comunicaciones Device-to-Device (D2D) sobre WiFi Direct sin comprometer el rendimiento de dispositivos IoT con recursos limitados.

La validación teórica y experimental del protocolo propuesto permite extraer tres conclusiones fundamentales:

1. **Resiliencia ante ataques de saturación:** La incorporación de un algoritmo de filtrado contextual en la fase de descubrimiento permite discriminar eficazmente el tráfico malicioso. A diferencia de los estándares actuales que procesan indiscriminadamente todas las solicitudes, nuestra propuesta mitiga los ataques de Denegación de Servicio (DoS) en una etapa temprana, [2] preservando la vida útil de la batería y la disponibilidad del servicio.
2. **Integridad criptográfica ligera:** La implementación del esquema de compromiso (Commit/Open) junto con la autenticación mutua (MAKE) ha demostrado ser una solución robusta contra ataques de intermediario (Man-in-the-Middle). [1] Al evitar el intercambio de claves en texto claro y garantizar la frescura de los mensajes mediante marcas de tiempo, se logra un canal seguro utilizando únicamente operaciones criptográficas ligeras (XOR, hash), ideales para el hardware restringido del IoT.
3. **Viabilidad operativa:** Los resultados confirman que la seguridad no debe estar reñida con la eficiencia. El protocolo mantiene una latencia mínima y una alta estabilidad bajo estrés, validando su idoneidad para entornos críticos donde la inmediatez y la veracidad de los datos son imperativas.

En definitiva, la solución presentada ofrece un equilibrio óptimo entre seguridad y rendimiento, estableciendo un marco de confianza necesario para el despliegue masivo de aplicaciones colaborativas en las ciudades del futuro.

REFERENCIAS

- [1] G. S. Gaba, G. Kumar, T.-H. Kim, H. Monga, y P. Kumar, "Secure Device-to-Device communications for 5G enabled Internet of Things applications," *Computer Communications*, vol. 169, pp. 114–128, Ene. 2021. DOI: <https://doi.org/10.1016/j.comcom.2021.01.010>
- [2] A. Hadiks, Y. Chen, F. Li, y B. Liu, "A study of stealthy denial-of-service attacks in Wi-Fi Direct device-to-device networks," en *2014 IEEE 11th Consumer Communications and Networking Conference (CCNC)*, IEEE, 2014, pp. 507–508. DOI: <https://doi.org/10.1109/CCNC.2014.6866617>
- [3] W. Shen, B. Yin, X. Cao, L. X. Cai, y Y. Cheng, "Secure device-to-device communications over WiFi Direct," *IEEE Network*, vol. 30, no. 5, pp. 4–9, 2016. DOI: <https://doi.org/10.1109/MNET.2016.7579020>
- [4] S. Viehböck, "Brute forcing Wi-Fi Protected Setup," *Wi-Fi Protected Setup*, vol. 9, 2011. Disponible en: https://sviehb.files.wordpress.com/2011/12/viehboeck_wps.pdf
- [5] M. Wang y Z. Yan, "A survey on security in D2D communications," *Mobile Networks and Applications*, vol. 22, no. 2, pp. 195–208, 2017. DOI: <https://doi.org/10.1007/s11036-016-0741-5>
- [6] A. S. Alfa, B. T. Maharaj, H. A. Ghazaleh, y B. Awoyemi, "The role of 5G and IoT in Smart Cities," en *Handbook of Smart Cities*, Springer, 2018, pp. 31–54. DOI: https://doi.org/10.1007/978-3-319-97271-8_2
- [7] F. Gago, S. Eslava, M. A. Herrera, "Manual de Usuario: Protocolo MAKE para IoT," Documentación técnica, archivo Guía Usuario.pdf (Adjunto), 2025.
- [8] F. Gago, S. Eslava, M. A. Herrera, "Simulador-SmartCity" Código fuente de la demostración práctica, archivo Simulador-SmartCity.zip (Adjunto), 2025.